our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Onchain Heroes

Valor

SECURITY REVIEW

Date: 27 April 2026

CONTENTS

1. **About Shieldify Security** **3**
2. **Disclaimer** **3**
3. **About Onchain Heroes – Valor** **3**
4. **Risk classification** **4**
   - 4.1 Impact 4
   - 4.2 Likelihood 4
5. **Security Review Summary** **4**
   - 5.1 Protocol Summary 5
   - 5.2 Scope 5
6. **Findings Summary** **5**
7. **Findings** **5**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Onchain Heroes - Valor

Onchain Heroes is a hybrid web2/web3 game. Valor is an in-game soft currency backed 1:1 by USDC at a rate of 100 VALOR = 1 USDC. Players deposit USDC into the ValorVault smart contract and are credited VALOR off-chain in a Postgres database.

Withdrawal Process: Player will convert VALOR back to USDC minus a 5% fee, enforced via a two-step on-chain process with a configurable time delay.

Token Interaction

The contract interacts exclusively with **USDC** (Circle's USD Coin) on Abstract Mainnet. USDC is a proxied ERC-20 with 6 decimals, an admin blacklist, and global pause capability.

User Flows

Deposit Flow

1. Player approves the ValorVault contract for USDC spending
2. Player calls deposit(amount)
3. Contract emits Deposited(depositor, usdcAmount, valorCredited) where valorCredited = amount * 100
4. Contract pulls USDC from the player into the vault
5. Backend listens for the event (via Alchemy webhook) and credits VALOR in Postgres

Withdrawal Flow (Two-Step)

1. Player requests withdrawal via the backend API (specifying VALOR amount)
2. Backend validates balance, deducts VALOR, generates EIP-712 signature, returns to player
3. **Step 1 — Initiate (on-chain):** Player calls initiateWithdrawal(grossAmount, deadline, minNetAmount, signature)
    - Contract verifies signature against stored signer
    - Contract calculates fee split at current BPS rates and locks all amounts into a PendingWithdrawal struct

- Contract increments nonce (consumes signature)
- Contract emits WithdrawalInitiated with the finalizeAfter timestamp
- Only one pending withdrawal per wallet — reverts if one already exists

4. **Delay period** (configurable, default 24h) — gives the team time to detect anomalies via on-chain WithdrawalInitiated events and pause if needed
5. **Step 2 — Finalize (on-chain):** Player calls finalizeWithdrawal() (no parameters)

   - Contract verifies the delay has passed
   - Contract transfers net USDC to player, prize pool share to prizePoolWallet, team share to teamWallet
   - Contract emits WithdrawalFinalized

Admin Cancellation

The owner can call cancelWithdrawal(address) to cancel any player's pending withdrawal. The back-end listens for the WithdrawalCancelled event and refunds the player's VALOR balance.

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 4 days, with a total of 64 hours dedicated by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying six Informational recommendations. They're related to non-critical design inconsistencies and best practice deviations.

The Onchain Heroes team has been highly responsive to the Shieldify research team's inquiries.

5.1 Protocol Summary

Project Name	Onchain Heroes - Valor
Repository	ak-valor
Type of Project	GameFi
Security Review Timeline	4 days
Review Commit Hash	81fafd977ee0fe4c486e6a3c8db73be9ba7006ba

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
src/world/ValorVault.sol	284
Total	**284**

6. Findings Summary

The following issues have been identified, sorted by their severity:

- **Info** issues: `6`

ID	Title	Severity	Status
[I-01]	`ValorVaultProxy.fallback()` Is `payable` Without Need	Info	Acknowledged
[I-02]	Custom Storage Slot Does Not Follow ERC-7201	Info	Acknowledged
[I-03]	`emergencyWithdraw()` Cannot Sweep Foreign ERC-20 Tokens	Info	Acknowledged
[I-04]	`paused` Semantics Mismatch: Deposits Are Blocked Despite "withdrawals only" Comment	Info	Acknowledged
[I-05]	`withdrawalDelay()` Has No Maximum Cap	Info	Acknowledged
[I-06]	Guardian Role Self-Renunciation Not Blocked Despite `renounceOwnership()` Being Explicitly Disabled	Info	Acknowledged

7. Findings

[I-01] `ValorVaultProxy.fallback()` Is `payable` Without Need

Severity

Informational Risk

Description

The proxy's catch-all fallback is declared `payable`, but no user or admin function in `ValorVault` accepts ETH. The only payable entry points reachable via the proxy are Solady's `upgradeToAndCall()` (payable purely for dispatch-gas reasons) and the overridden `renounceOwnership()`, which always reverts.

A `payable` fallback therefore provides no legitimate path for ETH and only turns operator mistakes (e.g. a stray `value` field on an upgrade tx) into stuck funds, because the implementation has no ETH handler and `emergencyWithdraw()` is ERC-20-only.

Location of Affected Code

File: src/world/ValorVaultProxy.sol#L42-L51

```
fallback() external payable {
    assembly {
        calldatacopy(0, 0, calldatasize())
        let result := delegatecall(gas(), sload(IMPLEMENTATION_SLOT), 0,
            calldatasize(), 0, 0)
        returndatacopy(0, 0, returndatasize())
        switch result
        case 0 { revert(0, returndatasize()) }
        default { return(0, returndatasize()) }
    }
}
```

Impact

Any value-bearing call that is otherwise valid (e.g. `upgradeToAndCall{value: x}(...)`) silently lands ETH on the proxy balance with no recovery path. Low probability, no direct user-fund impact, but a permanent leak when triggered.

Proof of Concept

```
// Operator upgrade script with a mistaken 'value' field:
proxy.call{value: 1 ether}(
    abi.encodeWithSignature("upgradeToAndCall(address,bytes)", newImpl, "
        ")
);
```

With `payable` removed, Solidity's value-check reverts the tx before any state change.

Recommendation

Drop `payable` from the proxy fallback:

```
fallback() external {
    assembly { ... }
```

```
}
```

Team Response

Acknowledged.

[I-02] Custom Storage Slot Does Not Follow ERC-7201

Severity

Informational Risk

Description

`ValorVault` uses a hand-rolled namespaced storage pattern instead of ERC-7201. The slot is derived from a single `keccak256` of a string, truncated to `uint72` (9 bytes) to save bytecode. The struct also lacks the `@custom:storage-location erc7201:<namespace>` NatSpec tag.

Deviations from ERC-7201: - No double-hash-with-`-1` (`keccak256(abi.encode(uint256(keccak256("erc7201:<ns`
). - No low-byte mask reserving 256 contiguous slots.

Without the NatSpec annotation, upgrade-safety tooling (OZ Upgrades Plugin, Slither, Foundry storage-diff) cannot auto-detect the namespaced struct, so every future upgrade becomes a manual layout review.

Location of Affected Code

File: src/world/ValorVault.sol#L92-L122

```
struct ValorVaultStorage {
    address token;
    address signer;
    ...
}
```

File: src/world/ValorVault.sol#L494-L500

```
function _getStorage() internal pure returns (ValorVaultStorage storage $
    ) {
    // Truncate to 9 bytes to reduce bytecode size.
    uint256 s = uint72(bytes9(keccak256("OCH_VALOR_VAULT_STORAGE")));
    assembly ("memory-safe") { $.slot := s }
}
```

Impact

- **No automated upgrade layout checks.** Tools key off `@custom:storage-location erc7201:...`; every upgrade PR must be hand-reviewed.

- **Silent drift risk.** The slot is recomputed from the string literal on each call. A typo or rename of the namespace string in a future implementation silently repoints storage to an empty root, bricking the proxy.
- **Reduced collision headroom.** A future second namespaced struct must manually verify no 72-bit collision — ERC-7201's full-width masked slot avoids this.

Proof of Concept

```
// v2 author accidentally changes the namespace string:
function _getStorage() internal pure returns (ValorVaultStorage storage $
    ) {
    uint256 s = uint72(bytes9(keccak256("OCH_VALOR_VAULT_STORAGES")))); //
        typo
    assembly ("memory-safe") { $.slot := s }
}
// After upgradeToAndCall(v2, ""): v2 reads/writes a fresh empty root.
// token(), signer(), fees return zero; pending withdrawals invisible.
// No tool catches this — there is no NatSpec to diff against.
```

Recommendation

Path A — Full ERC-7201 (preferred):

```
/// @custom:storage-location erc7201:onchainheroes.storage.ValorVault
struct ValorVaultStorage { ... }

// keccak256(abi.encode(uint256(keccak256("onchainheroes.storage.
    ValorVault")) - 1)) & ~bytes32(uint256(0xff))
bytes32 private constant _VALOR_VAULT_STORAGE = 0x...;

function _getStorage() internal pure returns (ValorVaultStorage storage $
    ) {
    assembly ("memory-safe") { $.slot := _VALOR_VAULT_STORAGE }
}
```

Path B — Keep 72-bit slot (if bytecode size is binding), but harden it:

```
/// @custom:storage-location erc7201:onchainheroes.storage.ValorVault
struct ValorVaultStorage { ... }

// Precomputed: uint72(bytes9(keccak256("OCH_VALOR_VAULT_STORAGE")))
uint256 private constant _VALOR_VAULT_STORAGE_SLOT = 0x...;

function _getStorage() internal pure returns (ValorVaultStorage storage $
    ) {
    assembly ("memory-safe") { $.slot := _VALOR_VAULT_STORAGE_SLOT }
}
```

Team Response

Acknowledged.

[I-03] `emergencyWithdraw()` Cannot Sweep Foreign ERC-20 Tokens

Severity

Informational Risk

Description

The vault rescue path is limited to the configured vault token only. In `emergencyWithdraw()`, the transfer target asset is hard-coded as `$.token`, so the owner cannot recover any other ERC-20 that is accidentally sent to the vault address.

This creates an operational recovery gap for foreign-token transfers (for example, accidental USDT/DAI transfers, airdrops, or integration mistakes). Even with owner access, those non-configured tokens remain trapped because no generic sweep function exists.

Location of Affected Code

File: src/world/ValorVault.sol#L377-L383

```
function emergencyWithdraw(address to, uint256 amount, string calldata
    reason) external onlyOwner {
     ValorVaultStorage storage $ = _getStorage();
     if (to == address(0)) revert ZeroAddress();
     if (amount == 0) revert InvalidAmount();
     SafeTransferLib.safeTransfer($.token, to, amount);
     emit EmergencyWithdrawal(to, amount, reason);
}
```

Impact

Non-configured ERC-20 tokens sent to the vault cannot be recovered.

Recommendation

```
function emergencyWithdraw(address tokenAddr, address to, uint256 amount)
     external onlyOwner {
     if (to == address(0)) revert ZeroAddress();
     SafeTransferLib.safeTransfer(tokenAddr, to, amount);
     emit Swept(tokenAddr, to, amount);
}
```

Team Response

Acknowledged.

[I-04] `paused` Semantics Mismatch: Deposits Are Blocked Despite "withdrawals only" Comment

Severity

Informational Risk

Description

The storage comment documents the pause flag as affecting withdrawals only, but `deposit()` is also guarded by `whenNotPaused`. This creates a specification-to-implementation mismatch: integrators, runbooks, and UI logic that trust the comment may assume deposits remain available during pause, while the contract actually reverts deposits with `Paused()`.

Location of Affected Code

File: src/world/ValorVault.sol#L188-L192

```
/// @dev Pause flag for emergency stops (withdrawals only)
bool paused;

modifier whenNotPaused() {
    ValorVaultStorage storage $ = _getStorage();
    if ($.paused) revert Paused();
    _;
}
```

File: src/world/ValorVault.sol#L201-L206

```
function deposit(uint256 amount) external whenNotPaused {
    ValorVaultStorage storage $ = _getStorage();
    if (amount == 0) revert InvalidAmount();
    emit Deposited(msg.sender, amount, amount * VALOR_RATE);
    SafeTransferLib.safeTransferFrom($.token, msg.sender, address(this),
        amount);
}
```

Impact

Off-chain assumptions can be wrong if integrators follow comments instead of runtime checks.

Recommendation

Align documented semantics with implemented behavior by choosing one policy and making both code and comments consistent:

Team Response

Acknowledged.

[I-05] `withdrawalDelay()` Has No Maximum Cap

Severity

Informational Risk

Description

An unbounded `_delay()` parameter lets the owner set an arbitrarily large delay, permanently locking pending withdrawals since the value is snapshotted at initiation time.

Location of Affected Code

File: src/world/ValorVault.sol#L366-L371

```
function setWithdrawalDelay(uint256 _delay) external onlyOwner {
    ValorVaultStorage storage $ = _getStorage();
    uint256 oldDelay = $.withdrawalDelay;
    $.withdrawalDelay = _delay;
    emit WithdrawalDelayUpdated(oldDelay, _delay);
}
```

File: src/world/ValorVault.sol#L400-L404

```
function canFinalizeWithdrawal(address account) external view returns (
   bool) {
    PendingWithdrawal memory pw = _getStorage().pendingWithdrawals[
       account];
    if (pw.grossAmount == 0) return false;
    return block.timestamp >= pw.initiatedAt + pw.delay;
}
```

Impact

Misconfiguration can set an impractically large delay and degrade withdrawal liveness for newly initiated withdrawals.

Recommendation

Add a hard maximum cap and validate it wherever delay is set (`constructor()`, `init()` and `setWithdrawalDelay()`).

```
error InvalidDelay();

uint256 private constant MAX_WITHDRAWAL_DELAY = 30 days;

function setWithdrawalDelay(uint256 _delay) external onlyOwner {
    if (_delay > MAX_WITHDRAWAL_DELAY) revert InvalidDelay();
    ValorVaultStorage storage $ = _getStorage();
    uint256 oldDelay = $.withdrawalDelay;
```

```
    $.withdrawalDelay = _delay;
    emit WithdrawalDelayUpdated(oldDelay, _delay);
}
```

Team Response

Acknowledged.

[I-06] Guardian Role Self-Renunciation Not Blocked Despite `renounceOwnership()` Being Explicitly Disabled

Severity

Informational Risk

Description

`ValorVault` overrides `renounceOwnership()` to permanently revert, establishing the design intent that privileged roles in this contract should not be self-removable. The same protection is not applied to `renounceRoles()` — inherited unchanged from Solady's `OwnableRoles` — so any address holding `GUARDIAN_ROLE` can silently drop its own role at any time without owner involvement.

This is an inconsistency between the contract's stated policy for the owner role and its implemented policy for the guardian role.

Location of Affected Code

File: src/world/ValorVault.sol#L468-L470

```
function renounceOwnership() public payable override {
    revert();
}

// renounceRoles is inherited from Solady OwnableRoles with no override —
// any GUARDIAN_ROLE holder can call it and silently drop the role.
```

Impact

A guardian can unilaterally remove its own pause capability with no confirmation, no cooldown, and no event tagged to `GUARDIAN_ROLE` specifically. The owner can re-grant the role, so there is no fund-loss path, but the inconsistency contradicts the design intent expressed by the `renounceOwnership()` override and silently removes a defence-in-depth layer until the owner notices and re-grants.

Recommendation

Override `renounceRoles()` to block self-removal of `GUARDIAN_ROLE`, mirroring the existing `renounceOwnership()` policy:

```
error CannotRenounceGuardian();

function renounceRoles(uint256 roles) public payable override {
    if (roles & GUARDIAN_ROLE != 0) revert CannotRenounceGuardian();
    super.renounceRoles(roles);
}
```

Guardian removals then route through `revokeRoles(guardian, GUARDIAN_ROLE)` which is already `onlyOwner`, keeping all privileged-role lifecycle changes under owner control.

Team Response

Acknowledged.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!